The Comprehensive Architecture of Object-Oriented Programming in C++: A Technical Treatise

The evolution of computational paradigms from procedural logic to object-oriented structures represents a fundamental shift in how software engineers conceptualize and implement complex systems. In the context of C++, this transition is not merely a syntactic variation but a radical reorientation toward data abstraction as the primary organizing principle of program logic.[1] While procedural programming centers on a sequential list of instructions and the creation of functions—or "procedures"—to perform operations on data objects passed into them, object-oriented programming (OOP) synthesizes properties and behaviors into cohesive units known as class objects.[2] This synthesis allows for the modeling of real-world entities with a degree of fidelity that procedural languages cannot easily achieve, providing a framework for modularity, extensibility, and robust code reusability.[2]

The Philosophical and Mathematical Genesis of Object Orientation

The conceptual roots of object-oriented programming in C++ are deeply intertwined with the mathematical definition of an object as a set combined with the operations permissible upon that set.[1] For instance, a set of integers is defined not just by its members but by the arithmetic operations (addition, subtraction, multiplication) that maintain the integrity of that set. In the same vein, programming skills have evolved to mirror these problem-solving patterns. Where a language might lack a built-in type like a complex number or a vector, the C++ class feature allows a programmer to extend the language with user-defined types that behave as if they were intrinsic to the language itself.[1]

This reorientation replaces code abstraction—the hallmark of structured programming—with data abstraction, comprising the pillars of encapsulation, inheritance, and polymorphism.[1] The transition is often necessitated by the realization that real-world entities are defined by an inseparable combination of state (properties) and action (behaviors).[2] A procedural approach separates these two, often leading to fragile code where data is exposed to unintended modifications and logic is scattered across disparate functions. OOP addresses this by creating program-defined types that manage their own internal state while exposing a well-defined interface for interaction.[2]

Feature	Procedural Programming	Object-Oriented Programming

Central Focus	Functions and program logic ("procedures").	Data types and their associated behaviors ("objects").
Logic-Data Relationship	Data is separate and passed to functions.	Data and functions are bundled into classes.
Model of Reality	Models actions as independent instructions.	Models reality as a collection of interacting entities.
Modularity	Achieved through function decomposition.	Achieved through class encapsulation and hierarchy.
Data Security	Low; data is often globally accessible or passed by reference.	High; data is hidden through access specifiers.

2

The Class as the Architectural Blueprint

In the C++ language specification, the class is the primary mechanism for implementing object-orientation. It serves as a user-defined template or blueprint for creating objects, defining the structure (attributes) and behavior (methods) that its instances will possess.[4] It is critical to distinguish between the class definition and the object instantiation. A class definition is a logical construct that describes what an object will consist of and the operations it can perform; however, it does not allocate memory.[5] Memory is only allocated when an object is instantiated—that is, when a real instance of the class is created during program execution.[4]

Syntax and Structural Definition

A class is defined using the class keyword, followed by an identifier and a body enclosed in curly braces, terminated by a semicolon.[6] Within this body, the programmer declares data members, which represent the properties or state of the object, and member functions, which define its behaviors.[6] For example, a Room class might encapsulate the physical dimensions of a space and provide methods for calculating its area and volume.[7]

C++

```cpp
#include <iostream>

class Room {
public:
    double length;
    double breadth;
    double height;

    // Member function to calculate area
    double calculate_area() {
        return length * breadth;
    }

    // Member function to calculate volume
    double calculate_volume() {
        return length * breadth * height;
    }
};

int main() {
    Room room1; // Object instantiation
    room1.length = 42.5;
    room1.breadth = 30.8;
    room1.height = 19.2;

    std::cout << "Area: " << room1.calculate_area() << std::endl;
    return 0;
}
```

7

In this architectural model, room1 is a concrete entity—an object—occupying memory and holding specific values for its attributes, whereas Room remains the conceptual framework.[5] This distinction is fundamental to understanding the lifecycle of C++ programs, where many objects can be generated from a single class blueprint, each maintaining its own unique state while sharing the same underlying logic.[4]

Class Invariants and State Integrity

A sophisticated aspect of class design in C++ is the maintenance of class invariants—conditions that must remain true throughout the object's lifetime to ensure it remains in a valid state.[8] For instance, a Fraction class might have a simple invariant: the denominator must never be zero.[8] In a procedural struct, maintaining this invariant is the responsibility of the caller, which can lead to undefined behavior if a developer forgets to validate the data.[8] In a class-based model, the class itself assumes this responsibility by hiding its data and providing controlled access, a concept known as encapsulation.[8]

Encapsulation: The Principle of Data Hiding and Controlled Access

Encapsulation is the process of bundling data and the functions that modify it into a single unit, thereby shielding the internal state from unwanted alterations.[9] This principle serves as the guardian of data privacy and integrity, ensuring that objects are modified only through a designated public interface.[10] In C++, encapsulation is achieved through access specifiers: public, private, and protected.[9]

Access Specifiers and Visibility Control

Access specifiers determine the accessibility of class members from different parts of the program.[12] By default, members of a C++ class are private, whereas members of a struct are public.[8]

1. **Private**: These members are accessible only within the class itself. They are hidden from external functions and even from derived classes.[9] This is the primary tool for data hiding, protecting sensitive internal implementations like bank balances or internal pointers.[9]
2. **Protected**: These members are accessible within the class and its derived classes (subclasses), but they remain hidden from the rest of the program.[12] This allows for a degree of flexibility in inheritance hierarchies while still maintaining a barrier against external misuse.[12]
3. **Public**: These members define the object's interface and are accessible from anywhere in the program.[9]

Access Specifier	Accessible within Class	Accessible in Derived Class	Accessible Externally
Private	Yes	No	No

Protected	Yes	Yes	No
Public	Yes	Yes	Yes

12

Mechanisms of Controlled Interaction: Getters and Setters

To allow interaction with private data while maintaining invariants, C++ utilizes getter and setter methods.[10] A getter method retrieves the value of a private attribute, while a setter method provides a controlled way to modify it, often including validation logic.[9] Consider a BankAccount class where the balance is private to prevent arbitrary modification.[9]

C++

```
class BankAccount {
private:
    double balance;

public:
    BankAccount(double initialBalance) : balance(initialBalance) {}

    // Getter for balance
    double getBalance() const {
        return balance;
    }

    // Setter (Deposit) with validation
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
};
```

10

This approach ensures that the internal state of the BankAccount object can only be changed

through valid transactions, preventing scenarios where a balance might be set to a negative value or altered without proper logging.[9]

Inheritance: Modeling Hierarchies and Specialization

Inheritance is a structural mechanism that allows a new class, known as the derived class or child class, to inherit the properties and behaviors of an existing class, referred to as the base class or parent class.[15] This creates a hierarchy that models "is-a" relationships—for instance, a Dog is-an Animal, or a SavingsAccount is-a BankAccount.[3]

Architectural Patterns of Inheritance

C++ provides a versatile set of inheritance models to represent various real-world and computational structures:

1. **Single Inheritance**: A derived class inherits from only one base class. This is the simplest form of hierarchy, enabling basic code reuse.[16]
2. **Multiple Inheritance**: A derived class inherits from more than one base class.[16] This allows an object to combine functionalities from multiple sources, such as a Smartphone inheriting from both Camera and Phone.[19]
3. **Multilevel Inheritance**: A class is derived from another derived class, creating a chain of inheritance (e.g., Animal → Mammal → Dog).[16]
4. **Hierarchical Inheritance**: Multiple derived classes share a single base class, allowing common functionality to be centralized in the parent.[16]
5. **Hybrid Inheritance**: A complex combination of the above patterns, often used in large-scale system designs.[15]

Inheritance Access Modes and Member Visibility

When a class inherits from another, the visibility of the base class's members in the derived class is determined by the inheritance access mode.[14]

Inheritance Mode	Base Public Becomes	Base Protected Becomes	Base Private Becomes
Public Inheritance	Public	Protected	Inaccessible
Protected Inheritance	Protected	Protected	Inaccessible
Private Inheritance	Private	Private	Inaccessible

15

Public inheritance is the most common form, as it preserves the "is-a" relationship and allows the derived class to be used wherever the base class is expected.[15] Private inheritance, conversely, is often used to model "is-implemented-in-terms-of" relationships, where the public interface of the base class is hidden from the users of the derived class.[14]

The Diamond Problem and the Role of Virtual Inheritance

Multiple inheritance, while powerful, introduces specific architectural challenges, the most notable of which is the "Diamond Problem".[19] This situation arises when a derived class inherits from two parent classes that themselves share a common base class.[19] Without intervention, the most-derived class would contain two distinct copies of the common base class's members, leading to ambiguity during member access and increased memory consumption.[20]

Ambiguity and Redundancy

If Parent1 and Parent2 both inherit from Base, and Child inherits from both Parent1 and Parent2, a call to a method defined in Base through a Child object would be ambiguous.[19] The compiler cannot determine whether to use the version of the base member inherited through Parent1 or the one through Parent2.[20]

Virtual Base Classes as a Solution

C++ resolves this through virtual inheritance.[19] By declaring the base class as virtual during inheritance, the programmer ensures that only one shared instance of the base class exists within the derived object's memory layout, regardless of how many inheritance paths lead to it.[19]

C++

```
class Base {
public:
    void display() { std::cout << "Base class" << std::endl; }
};

class Parent1 : virtual public Base {};
```

```
class Parent2 : virtual public Base {};

class Child : public Parent1, public Parent2 {};

int main() {
    Child obj;
    obj.display(); // Unambiguous thanks to virtual inheritance
    return 0;
}
```

[19]

While virtual inheritance solves the diamond problem, it introduces a minor performance overhead and complexity in constructor initialization, as the most-derived class becomes responsible for initializing the virtual base.[19]

Polymorphism: Static and Dynamic Resolution

Polymorphism, derived from the Greek for "many forms," is the capability of a single interface to represent different underlying forms or types.[4] In C++, this is bifurcated into compile-time (static) polymorphism and run-time (dynamic) polymorphism.[23]

Compile-Time Polymorphism: Static Binding

Compile-time polymorphism is resolved during the compilation phase.[23] It is primarily achieved through function overloading and operator overloading.[23] The compiler examines the function signature—the name and types of arguments—to determine which specific implementation to invoke.[23]

Operator overloading is a sophisticated form of static polymorphism where standard C++ operators (such as +, -, or *) are given new meanings for user-defined types.[25] For example, a Complex number class can overload the + operator to perform mathematical addition of complex numbers rather than simple integer addition.[25]

C++

```
class Complex {
private:
    float real, imag;
public:
```

```
    Complex(float r = 0, float i = 0) : real(r), imag(i) {}

    // Overloading the + operator
    friend Complex operator+(const Complex& c1, const Complex& c2) {
        return Complex(c1.real + c2.real, c1.imag + c2.imag);
    }
};
```

[25]

Run-Time Polymorphism: Dynamic Binding

Run-time polymorphism allows a program to decide which function to call while it is running, based on the actual type of the object being pointed to.[23] This is achieved using virtual functions and inheritance.[23] When a base class pointer or reference is used to call a virtual function, C++ performs dynamic dispatch to ensure the derived class's version of the function is executed.[23]

Attribute	Compile-Time Polymorphism	Run-Time Polymorphism
Commonly Known As	Static Binding / Early Binding.	Dynamic Binding / Late Binding.
Mechanisms	Function & Operator Overloading.	Virtual Functions & Overriding.
Decision Time	At the time of compilation.	During program execution.
Performance	Faster (no vtable lookups).	Slower (requires vtable indirection).
Flexibility	Lower; types must be known.	Higher; enables generic programming.

[23]

The Virtual Function Table (Vtable) Mechanics

The underlying implementation of dynamic polymorphism usually involves a Virtual Function Table (vtable).[27] For every class containing virtual functions, the compiler generates a vtable

that stores pointers to the virtual functions for that class.[27] Each object of that class contains a hidden pointer, the vptr, which points to the vtable.[27] When a virtual function is called through a pointer, the program follows the vptr to the vtable to find the correct function address for that specific object type.[27]

Abstraction and the Design of Robust Interfaces

Abstraction is the design principle of simplifying complex systems by modeling classes based on their essential qualities and behaviors while hiding unnecessary implementation details.[4] It provides a high-level view of what an object does, rather than how it does it.[4] In C++, abstraction is formalized through abstract classes and pure virtual functions.[29]

Pure Virtual Functions and Abstract Classes

A pure virtual function is declared by assigning = 0 to its signature within the class declaration.[29] This syntax indicates that the base class does not provide an implementation for that function.[27] A class that contains at least one pure virtual function is an abstract class.[26]

Abstract classes cannot be instantiated—meaning the programmer cannot create an object of that class type.[26] Instead, they serve as foundational blueprints for derived classes.[29] To become a "concrete" class that can be instantiated, a derived class must override every pure virtual function inherited from the abstract parent.[27]

C++

```
class Shape {
public:
    // Pure virtual function
    virtual double calculateArea() = 0;

    // Virtual destructor for safe cleanup
    virtual ~Shape() {}
};

class Circle : public Shape {
private:
    double radius;
public:
```

```
    Circle(double r) : radius(r) {}
    double calculateArea() override {
        return 3.14159 * radius * radius;
    }
};
```

29

Interface Classes and Architectural Contracts

While C++ does not have a native interface keyword, a class where all member functions are pure virtual functions effectively acts as an interface.[26] This is a powerful design pattern for creating contracts that ensure all subclasses implement a specific set of behaviors without dictating their internal implementation.[26] This facilitates loose coupling between system components, making code more maintainable and easier to test.[26]

The Lifecycle of Objects: Constructors and Destructors

Managing the lifecycle of an object—its birth, active state, and cleanup—is a critical responsibility in C++, particularly when dealing with manual resource management.[33]

Constructors: Initializing State

A constructor is a special member function that is automatically executed when an object is created.[33] It is primarily used to initialize data members and acquire necessary resources, such as memory or file handles.[33]

- **Default Constructor**: A constructor that takes no arguments. If a programmer defines no constructors, the compiler may generate a default one.[34]
- **Parameterized Constructor**: Allows the caller to specify initial values for the object's properties.[25]
- **Copy Constructor**: Creates a new object as a copy of an existing object.[34]

In an inheritance hierarchy, constructors are called in a top-down fashion: the base class constructor runs first, followed by the constructors of the derived classes.[15] This ensures that the foundation of the object is ready before the specialized parts are added.[16]

Destructors: Resource Reclamation

A destructor is called when an object goes out of scope or is explicitly deleted.[33] It is responsible for cleanup tasks, such as freeing memory and closing file descriptors.[33] Destructors follow a bottom-up execution order in inheritance: the derived class destructor

runs first, then the parent class destructor.[16]

The Necessity of Virtual Destructors in Polymorphism

A critical rule in C++ development is that base classes in a polymorphic hierarchy must have a virtual destructor.[37] If an object is deleted through a base class pointer and the destructor is not virtual, the program will only execute the base class destructor, potentially leaving the resources of the derived class un-reclaimed.[28] This is a common source of memory leaks in complex C++ applications.[38]

Resource Acquisition Is Initialization (RAII) and Modern C++

The RAII idiom is a fundamental C++ design principle that ties resource management to the lifetime of stack-allocated objects.[35] In this model, a resource (like a memory block, a mutex lock, or a network connection) is acquired in the object's constructor and automatically released in its destructor.[35]

Exception Safety and Reliable Cleanup

The power of RAII lies in its ability to ensure resource cleanup even in the presence of exceptions.[35] Since the C++ language guarantees that destructors of stack-allocated objects will be called as the stack is unwound during an exception, the programmer does not need to manually call "close" or "free" functions.[34]

Standard Library tools that utilize RAII include:

- **Smart Pointers**: std::unique_ptr and std::shared_ptr manage heap memory by automatically deleting the pointed-to object when the pointer itself is destroyed.[36]
- **Lock Guards**: std::lock_guard acquisitions a mutex in its constructor and releases it in its destructor, preventing deadlocks if a function exits prematurely.[35]

C++

```
#include <memory>
#include <iostream>

class Resource {
public:
    Resource() { std::cout << "Resource Acquired\n"; }
```

```
    ~Resource() { std::cout << "Resource Released\n"; }
};

void process() {
    // RAII via smart pointer
    std::unique_ptr<Resource> ptr = std::make_unique<Resource>();
    // Resource will be released automatically when 'process' returns
}
```

[42]

Deep vs. Shallow Copying: Navigating Memory Management

Copying objects in C++ requires careful consideration of how data, specifically dynamically allocated memory, is duplicated.[44]

Shallow Copy: The Member-wise Approach

A shallow copy duplicates all member variables exactly as they are.[44] For members that are raw pointers, this means the copy and the original share the same memory address.[44] This is often dangerous: if one object modifies the data or deletes it, the other object is left with a dangling pointer or corrupted state.[44] The default copy constructor generated by the compiler performs a shallow copy of pointers.[36]

Deep Copy: Independent Duplication

In a deep copy, the program allocates a new block of memory for the copy and duplicates the actual data pointed to by the original object.[44] This ensures that each object owns its unique resources, preventing cross-contamination and double-free errors.[44]

Feature	Shallow Copy	Deep Copy
Logic	Copies values of member variables (including addresses).	Copies values and allocates new memory for pointed-to data.
Independence	Original and copy share dynamic resources.	Original and copy are entirely independent.
Risk	High; leads to	Low; resources are safely

	double-deletion or dangling pointers.	partitioned.
Implementation	Default compiler behavior.	Requires custom copy constructor and assignment operator.

44

Advanced Object Relationships: Composition and Aggregation

While inheritance models "is-a" relationships, other patterns are necessary to describe how objects are composed into larger systems.[3]

Composition: "Part-of" Relationships

In composition, one object contains another object as a member variable, and the life of the "part" is tied to the life of the "whole".[3] For example, a Car object might contain an Engine object. When the Car is destroyed, the Engine is destroyed with it.[3]

Aggregation: "Member-of" Relationships

Aggregation is a weaker form of containment where the "part" can exist independently of the "whole".[3] For example, a Department might have several Professor objects. If the Department is disbanded, the Professors still exist.[3]

The Rule of Three, Five, and Zero

To manage these complex relationships safely, C++ developers often follow the Rule of Three/Five. If a class requires a custom version of any of the following, it likely needs custom versions of all of them to ensure correct behavior during copying and destruction [34]:

1. Destructor
2. Copy Constructor
3. Copy Assignment Operator
4. Move Constructor (C++11 and later)
5. Move Assignment Operator (C++11 and later)

Alternatively, the Rule of Zero suggests designing classes to use components (like std::string or std::vector) that handle their own resource management, allowing the compiler to generate correct defaults for the containing class.[34]

Conclusion: Synthesizing Paradigms for Scalable Architecture

The object-oriented programming model in C++ provides a robust framework for managing the inherent complexity of modern software development. By leveraging classes and objects, developers can transition from procedural logic to a more intuitive, entity-centric design that mirrors the structure of real-world problems.[1] Encapsulation ensures that data remains secure and invariants are maintained, while inheritance and polymorphism provide the flexibility to build extensible hierarchies and generic interfaces.[4]

Furthermore, the integration of deterministic destruction and RAII makes C++ uniquely suited for high-performance systems where resource management is paramount.[35] As software systems continue to grow in scale and complexity, the disciplined application of these OOP principles remains the cornerstone of professional C++ engineering, enabling the creation of code that is not only efficient but also maintainable, testable, and resilient to change.

Works cited

1. Classes - cppreference.com, accessed on January 31, 2026, https://cppreference.com/book/classes.html
2. 14.1 — Introduction to object-oriented programming - Learn C++, accessed on January 31, 2026, https://www.learncpp.com/cpp-tutorial/introduction-to-object-oriented-programming/
3. 23.1 — Object relationships - Learn C++, accessed on January 31, 2026, https://www.learncpp.com/cpp-tutorial/object-relationships/
4. C++ Classes and Objects - Great Learning, accessed on January 31, 2026, https://www.mygreatlearning.com/blog/class-in-c/
5. C++ Classes and Objects - GeeksforGeeks, accessed on January 31, 2026, https://www.geeksforgeeks.org/cpp/c-classes-and-objects/
6. Classes and Objects in C++ (With Examples) - WsCube Tech, accessed on January 31, 2026, https://www.wscubetech.com/resources/cpp/classes-objects
7. C++ Classes and Objects (With Examples) - Programiz, accessed on January 31, 2026, https://www.programiz.com/cpp-programming/object-class
8. 14.2 — Introduction to classes - Learn C++, accessed on January 31, 2026, https://www.learncpp.com/cpp-tutorial/introduction-to-classes/
9. Encapsulation In C++ With Detailed Examples (+Getter/ Setter) // Unstop, accessed on January 31, 2026, https://unstop.com/blog/encapsulation-in-cpp
10. Encapsulation in C++: A Guide to Data Privacy | CodeSignal Learn, accessed on January 31, 2026, https://codesignal.com/learn/courses/revisiting-software-design-patterns-in-cpp/lessons/encapsulation-in-cpp-a-guide-to-data-privacy
11. Encapsulation in C++ (Uses, Benefits With Examples) - WsCube Tech, accessed on January 31, 2026, https://www.wscubetech.com/resources/cpp/encapsulation

12. C++ Access Specifiers: Private, Public, and Protected Explained | by ..., accessed on January 31, 2026, https://devabdilah.medium.com/%EF%B8%8F-c-access-specifiers-private-public-and-protected-explained-c007cb9a1f40
13. Encapsulation, access specifiers (public, private, protected) | HNG Learn, accessed on January 31, 2026, https://hng.tech/learn/learn-c-plus-plus-programming-online-with-step-by-step-video-tutorials/encapsulation-access-specifiers-public-private-protected
14. Inheritance (C++) - Microsoft Learn, accessed on January 31, 2026, https://learn.microsoft.com/en-us/cpp/cpp/inheritance-cpp?view=msvc-170
15. Inheritance In C++ With Example Programs - Aticleworld, accessed on January 31, 2026, https://aticleworld.com/inheritance-in-c/
16. Learn Intermediate C++: Inheritance in C++ Cheatsheet - Codecademy, accessed on January 31, 2026, https://www.codecademy.com/learn/learn-intermediate-c-plus-plus/modules/inheritance-in-c-plus-plus/cheatsheet
17. Inheritance | C++ Tutorial - CodeWithHarry, accessed on January 31, 2026, https://www.codewithharry.com/tutorial/cpp-inheritance
18. Explore the 5 Types of Inheritance in C++ with Syntax & Example - DataFlair, accessed on January 31, 2026, https://data-flair.training/blogs/inheritance-in-cpp/
19. Understanding Virtual Inheritance and the Diamond Problem in C++ | by Atharv Mishra, accessed on January 31, 2026, https://medium.com/@antilogatharv/understanding-virtual-inheritance-and-the-diamond-problem-in-c-da7c63d76723
20. Multiple Inheritance in C++ - TutorialsPoint, accessed on January 31, 2026, https://www.tutorialspoint.com/cplusplus/cpp_multiple_inheritance.htm
21. Diamond Problem in C++ - GeeksforGeeks, accessed on January 31, 2026, https://www.geeksforgeeks.org/cpp/diamond-problem-in-cpp/
22. C++ OOP The Diamond Problem and Solving It Using Virtual Inheritance - SimplifyC++, accessed on January 31, 2026, https://www.simplifycpp.org/?id=a0223
23. Difference Between Compile Time and Run Time With Examples, accessed on January 31, 2026, https://www.almabetter.com/bytes/articles/difference-between-compile-time-and-run-time
24. Runtime polymorphism vs. compile time polymorphism in games? : r/cpp - Reddit, accessed on January 31, 2026, https://www.reddit.com/r/cpp/comments/1cj3gim/runtime_polymorphism_vs_compile_time_polymorphism/
25. C++ Operator Overloading (With Examples) - Programiz, accessed on January 31, 2026, https://www.programiz.com/cpp-programming/operator-overloading
26. Mastering C++ Abstract Class, Interface, and (Pure)Virtual Functions in Few Steps - Medium, accessed on January 31, 2026, https://medium.com/javarevisited/mastering-c-abstract-class-interface-and-pure-virtual-functions-in-few-steps-ddbeff770d1f

27. C++ - Pure Virtual Functions and Abstract Classes - TutorialsPoint, accessed on January 31, 2026, https://www.tutorialspoint.com/cplusplus/cpp_pure_virtual_functions_vs_abstract_classes.htm
28. Virtual Destructors in C++. Necessity, Good Practice, Bad Practice - Bulldogjob, accessed on January 31, 2026, https://bulldogjob.com/readme/virtual-destructors-in-c-necessity-good-practice-bad-practice
29. C++ Abstract Class and Pure Virtual Function - Programiz, accessed on January 31, 2026, https://www.programiz.com/cpp-programming/pure-virtual-funtion
30. Abstract classes (C++ only) - IBM, accessed on January 31, 2026, https://www.ibm.com/docs/en/i/7.4.0?topic=only-abstract-classes-c
31. 12.5. Pure Virtual Functions and Abstract Classes, accessed on January 31, 2026, https://icarus.cs.weber.edu/~dab/cs1410/textbook/12.Polymorphism/virtandabstract.html
32. How can I avoid the Diamond of Death when using multiple inheritance? - Stack Overflow, accessed on January 31, 2026, https://stackoverflow.com/questions/137282/how-can-i-avoid-the-diamond-of-death-when-using-multiple-inheritance
33. Overview of constructors and destructors (C++ only) - IBM, accessed on January 31, 2026, https://www.ibm.com/docs/en/xl-c-and-cpp-aix/16.1.0?topic=only-overview-constructors-destructors-c
34. C++ Destructors and RAII | hacking C++, accessed on January 31, 2026, https://hackingcpp.com/cpp/lang/destructors.html
35. Resource acquisition is initialization - Wikipedia, accessed on January 31, 2026, https://en.wikipedia.org/wiki/Resource_acquisition_is_initialization
36. Default copy constructor performs shallow or deep copy?? : r/cpp_questions - Reddit, accessed on January 31, 2026, https://www.reddit.com/r/cpp_questions/comments/1izd5ow/default_copy_constructor_performs_shallow_or_deep/
37. Understanding Virtual Destructors in C++ | by Kalu U. Ogbureke - Medium, accessed on January 31, 2026, https://medium.com/@udo.kalu/understanding-virtual-destructors-in-c-7d1ed3b6c2c2
38. When to Use Virtual Destructors in C++? - GeeksforGeeks, accessed on January 31, 2026, https://www.geeksforgeeks.org/cpp/when-to-use-virtual-destructors-in-cpp/
39. Virtual Destructor - GeeksforGeeks, accessed on January 31, 2026, https://www.geeksforgeeks.org/virtual-destructor/
40. c++ - When to use virtual destructors? - Stack Overflow, accessed on January 31, 2026, https://stackoverflow.com/questions/461203/when-to-use-virtual-destructors
41. virtual destructors vs normal destructor - c++ - Stack Overflow, accessed on January 31, 2026,

https://stackoverflow.com/questions/42779141/virtual-destructors-vs-normal-destructor
42. cpp_tutorials/docs/RAII.md at master - GitHub, accessed on January 31, 2026, https://github.com/behnamasadi/cpp_tutorials/blob/master/docs/RAII.md
43. RAII in Modern C++: Automatic Resource Management Done Right | by Wadix Technologies, accessed on January 31, 2026, https://medium.com/embedworld/raii-in-modern-c-automatic-resource-management-done-right-8da38e38f303
44. Shallow Copy and Deep Copy in C++ - GeeksforGeeks, accessed on January 31, 2026, https://www.geeksforgeeks.org/cpp/shallow-copy-and-deep-copy-in-c/
45. Deep copy vs Shallow Copy - c++ - Stack Overflow, accessed on January 31, 2026, https://stackoverflow.com/questions/2657810/deep-copy-vs-shallow-copy
46. Shallow and deep destructors? - Stack Overflow, accessed on January 31, 2026, https://stackoverflow.com/questions/57179851/shallow-and-deep-destructors